

Machine Learning for Mechanics

Lecture 3: Linear regression

By

Prof. Arpit Agrawal and Prof. Rajdip Nayek

Assistant Professor,
Department of Applied Mechanics,
IIT Delhi

IIT Delhi × Bechtel AI/ML Programme | 24 June 2026

Email: arpit@am.iitd.ac.in & rajdipn@am.iitd.ac.in

Webpage: **<https://rajdip-iitd.github.io/bechtel2/>**

Linear Regression

- In both regression and classification settings, we seek a function $f(\mathbf{x}^*)$ that maps the test input $\mathbf{x}^*$ to a prediction
- Regression $\rightarrow$ learn relationships between some input variables $\mathbf{x} = [x_1 \ x_2 \ \dots \ x_p]^T$ and a numerical output y
- The inputs can be either categorical or numerical, but let's consider that all p inputs are numerical

- Mathematically, regression is about learning a *model* f that maps the input to the output

$$y = f(\mathbf{x}) + \epsilon$$

- ϵ is an error term that describes everything about the input-output relationship that cannot be captured by the model
- From a statistical perspective, ϵ is considered as a **random variable** and referred to as **noise**, that is independent of $\mathbf{x}$ and has zero mean

Linear Regression

- **Linear regression model:** Output y (a scalar) is an affine combination of p input variables $x_1, x_2, \dots, x_p$ plus a noise term

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p + \epsilon$$

- $\theta_0, \theta_1, \theta_2, \dots, \theta_p$ are called the *parameters* of the model
- **Linear regression model:** Output y (a scalar) is an affine combination of $p + 1$ input variables $1, x_1, x_2, \dots, x_p$ plus a noise term

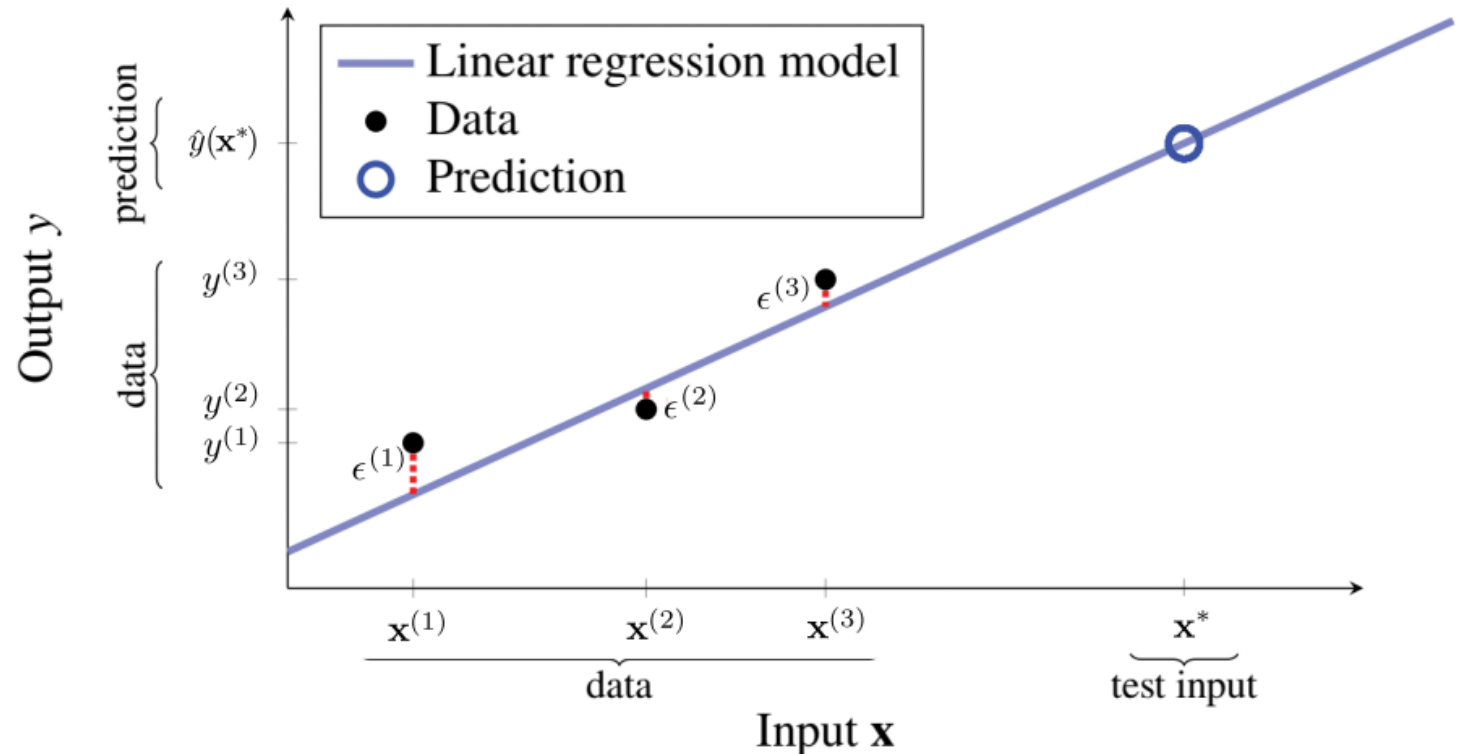
$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p + \epsilon = \begin{bmatrix} 1 & x_1 & \dots & x_p \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix} + \epsilon = \mathbf{x}^T \boldsymbol{\theta} + \epsilon$$

Linear Regression

- The linear regression model is a **parametric** function of the form $f(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta} + \epsilon$
- The parameters $\boldsymbol{\theta}$ can take arbitrary values, and the actual values that we assign to them will control the input–output relationship described by the model
- **Learning of the model** → finding suitable values for $\boldsymbol{\theta}$ based on observed training data

How to predict on test set?

- How to make predictions $\hat{y}(\mathbf{x}^*)$ for new previously unseen test inputs $\mathbf{x}^* = [1 \quad x_1^* \quad x_2^* \quad \dots \quad x_p^*]^T$?
- Let $\hat{\boldsymbol{\theta}}$ be the learned parameter value for the linear regression model
- Since the noise term ϵ is random with zero mean and independent of all observed variables, **we replace ϵ with 0 in the prediction**
- Prediction takes form: $\hat{y}(\mathbf{x}^*) = (\mathbf{x}^*)^T \hat{\boldsymbol{\theta}}$



Training a linear regression model from training data

- Training data: $\mathcal{T} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N$

$$\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\epsilon}, \quad \mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \mathbf{x}^{(2)T} \\ \vdots \\ \mathbf{x}^{(N)T} \end{bmatrix}, \quad \mathbf{x}^{(i)} = \begin{bmatrix} 1 \\ x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_p^{(i)} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix}$$

Training a linear regression model from training data

- Predicted outputs for training data, $\hat{\mathbf{y}} = [\hat{y}(\mathbf{x}^{(1)}) \quad \hat{y}(\mathbf{x}^{(2)}) \quad \dots \quad \hat{y}(\mathbf{x}^{(N)})]^T$
$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$$
- Learning the unknown parameters $\boldsymbol{\theta}$ amounts to finding their values such that $\hat{\mathbf{y}}$ is “similar” to $\mathbf{y}$
 - “Similar” $\rightarrow$ finding $\boldsymbol{\theta}$ such that $\hat{\mathbf{y}} - \mathbf{y} = \boldsymbol{\epsilon}$ is small
- Formulate a loss function, which gives a mathematical meaning to “similarity” between $\hat{\mathbf{y}}$ and $\mathbf{y}$

How to define the problem of learning model parameters?

- Use **loss function** $L(y, \hat{y}) \rightarrow$ measures the closeness of the model's prediction $\hat{y}$ to the observed data y
 - Smaller the loss, better the model fits the data, and vice versa
- Define **average loss (or cost function)** function, $J(\boldsymbol{\theta})$, as the average loss over the training data

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N L(y^{(i)}, \hat{y}(\mathbf{x}^{(i)}; \boldsymbol{\theta}))$$

- **Training a model** $\rightarrow$ finding the model parameters $\boldsymbol{\theta}$ that minimize the average training loss

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} J(\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N L(y^{(i)}, \hat{y}(\mathbf{x}^{(i)}; \boldsymbol{\theta}))$$

- $\hat{y}(\mathbf{x}^{(i)}; \boldsymbol{\theta})$ is the **model prediction** for the $\mathbf{x}^{(i)}$ training input and $y^{(i)}$ is the corresponding training output
 - The parameter $\boldsymbol{\theta}$ has been put as an argument to denote the dependence of the prediction on it
- The operator $\underset{\boldsymbol{\theta}}{\operatorname{argmin}}$ means 'the value of $\boldsymbol{\theta}$ for which the averaged loss function attains its minimum'

Least squares problem

- For regression, a commonly used loss function is the *squared error loss*

$$L(y, \hat{y}(\mathbf{x}; \boldsymbol{\theta})) = (y - \hat{y}(\mathbf{x}; \boldsymbol{\theta}))^2$$

- This loss function *grows quadratically* fast as the difference $(y - \hat{y}(\mathbf{x}; \boldsymbol{\theta}))$ increases

- The corresponding average loss function (or *cost function*)

$$J(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}(\mathbf{x}^{(i)}; \boldsymbol{\theta}))^2 = \frac{1}{N} \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 = \frac{1}{N} \|\boldsymbol{\epsilon}\|_2^2$$

- Here, $\|\cdot\|_2^2$ denotes the square of the Euclidean norm. Due to the square, it is called the *least squares cost function*

- In linear regression, the learning problem effectively *finds the best parameter estimate*

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N (y^{(i)} - (\mathbf{x}^{(i)})^T \boldsymbol{\theta})^2 = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \frac{1}{N} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$$

- Closed-form* solution exists $\rightarrow \hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ if $\mathbf{X}^T \mathbf{X}$ is invertible.

Linear regression algorithm

- Linear regression with squared error loss is very common in practice, due to [its closed-form solution](#)
- [Other loss functions lead to optimization problems](#) and often lack closed-form solutions

Training using linear regression model

Training Data: $\mathcal{T} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$

Result: Learned parameter vector $\hat{\boldsymbol{\theta}}$

1. Construct matrix of input features $\mathbf{X}$ and output vector $\mathbf{y}$
2. Compute $\hat{\boldsymbol{\theta}}$ by solving $(\mathbf{X}^T \mathbf{X}) \hat{\boldsymbol{\theta}} = \mathbf{X}^T \mathbf{y}$

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \mathbf{x}^{(2)T} \\ \vdots \\ \mathbf{x}^{(N)T} \end{bmatrix}, \quad \mathbf{x}^{(i)} = \begin{bmatrix} 1 \\ x_1^{(i)} \\ x_2^{(i)} \\ \vdots \\ x_p^{(i)} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix}$$

Testing using linear regression model

Data: Learned parameter vector $\hat{\boldsymbol{\theta}}$

Result: Prediction $\hat{y}(\mathbf{x}^*)$

1. Compute $\hat{y}(\mathbf{x}^*) = (\mathbf{x}^*)^T \hat{\boldsymbol{\theta}}$

A maximum likelihood perspective of least squares

- “**Likelihood**” refers to a statistical concept of a certain function which describes how likely is that a certain value of θ has generated the measurements $\mathbf{y}$
- Instead of selecting a loss function, one could start with the problem

$$\hat{\theta} = \operatorname{argmax}_{\theta} p(\mathbf{y}|\mathbf{X}; \theta)$$

- $p(\mathbf{y}|\mathbf{X}; \theta)$ is the probability density of all observed outputs $\mathbf{y}$ in the training data, given all inputs $\mathbf{X}$ and parameters θ
- $p(\mathbf{y}|\mathbf{X}; \theta)$ determines mathematically what ‘likely’ means

A maximum likelihood perspective of least squares

- **Common assumption:** Noise terms are independent and identically distributed (i.i.d.), each with a Gaussian distribution (also known as a normal distribution) with mean zero and variance σ_ϵ^2

$$\epsilon \sim \mathcal{N}(\epsilon; 0, \sigma_\epsilon^2)$$

- Implies that all observed training data points are independent, and $p(\mathbf{y}|\mathbf{X}; \boldsymbol{\theta})$ factorizes out as

$$p(\mathbf{y}|\mathbf{X}; \boldsymbol{\theta}) = \prod_{i=1}^N p(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta})$$

- The linear regression model, $y = \mathbf{x}^T \boldsymbol{\theta} + \epsilon$, together with i.i.d. Gaussian noise assumption leads to

$$p(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}) = \mathcal{N}(y^{(i)}; (\mathbf{x}^{(i)})^T \boldsymbol{\theta}, \sigma_\epsilon^2) = \frac{1}{\sqrt{2\pi\sigma_\epsilon^2}} \exp\left(-\frac{1}{2\sigma_\epsilon^2} (y^{(i)} - (\mathbf{x}^{(i)})^T \boldsymbol{\theta})^2\right)$$

- Recall, we want to maximize the likelihood w.r.t. the parameter $\boldsymbol{\theta}$
- Better to work with logarithm of the likelihood (log-likelihood) for numerical reasons

$$\ln p(\mathbf{y}|\mathbf{X}; \boldsymbol{\theta}) = \sum_{i=1}^N \ln(p(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}))$$

A maximum likelihood perspective of least squares

- Better to work with logarithm of the likelihood (**log-likelihood**) to prevent numerical overflow

$$\ln p(\mathbf{y}|\mathbf{X}; \boldsymbol{\theta}) = \sum_{i=1}^N \ln \left(p(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}) \right)$$

- **Logarithm is a monotonically increasing** function, maximising the loglikelihood is equivalent to maximising the likelihood

- The linear regression model, $y = \mathbf{x}^T \boldsymbol{\theta} + \epsilon$, together with i.i.d. Gaussian noise assumption leads to

$$\ln p(\mathbf{y}|\mathbf{X}; \boldsymbol{\theta}) = -\frac{N}{2} \ln(2\pi\sigma_\epsilon^2) - \frac{1}{2\sigma_\epsilon^2} \sum_{i=1}^N \left(y^{(i)} - (\mathbf{x}^{(i)})^T \boldsymbol{\theta} \right)^2$$

$$\hat{\boldsymbol{\theta}} = \operatorname{argmax}_{\boldsymbol{\theta}} \ln p(\mathbf{y}|\mathbf{X}; \boldsymbol{\theta}) = \operatorname{argmax}_{\boldsymbol{\theta}} \left(-\sum_{i=1}^N \left(y^{(i)} - (\mathbf{x}^{(i)})^T \boldsymbol{\theta} \right)^2 \right) = \operatorname{argmin}_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^N \left(y^{(i)} - (\mathbf{x}^{(i)})^T \boldsymbol{\theta} \right)^2$$

- Recall the same estimate is also obtained from linear regression with the least squares cost
- Using **squared error loss is equivalent to assuming a Gaussian noise distribution** in maximum likelihood formulation
- Other assumptions on ϵ lead to other loss functions (will discuss later)

Polynomial Regression

- Linear regression is rigid and not flexible – they can fit only straight lines.
- Make linear regression more flexible by increasing the input dimension p .
- **Question:** How to increase input dimension?
- **Common approach:** Add non-linear transformation of the input.
- **Polynomial Regression:** $y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \dots + \theta_p x^p + \epsilon$

Polynomial Regression

— Recall $y = \underline{x}^T \underline{\theta}$ where $\underline{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix}$, $\underline{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{bmatrix}$

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \dots + \theta_p x^p + \epsilon$$

Polynomial regression

— If $x_1 = x$, $x_2 = x^2$, $x_3 = x^3$, ..., $x_p = x^p \Rightarrow y = \begin{bmatrix} 1 & x & x^2 & x^3 & \dots & x^p \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \\ \vdots \\ \theta_p \end{bmatrix}$

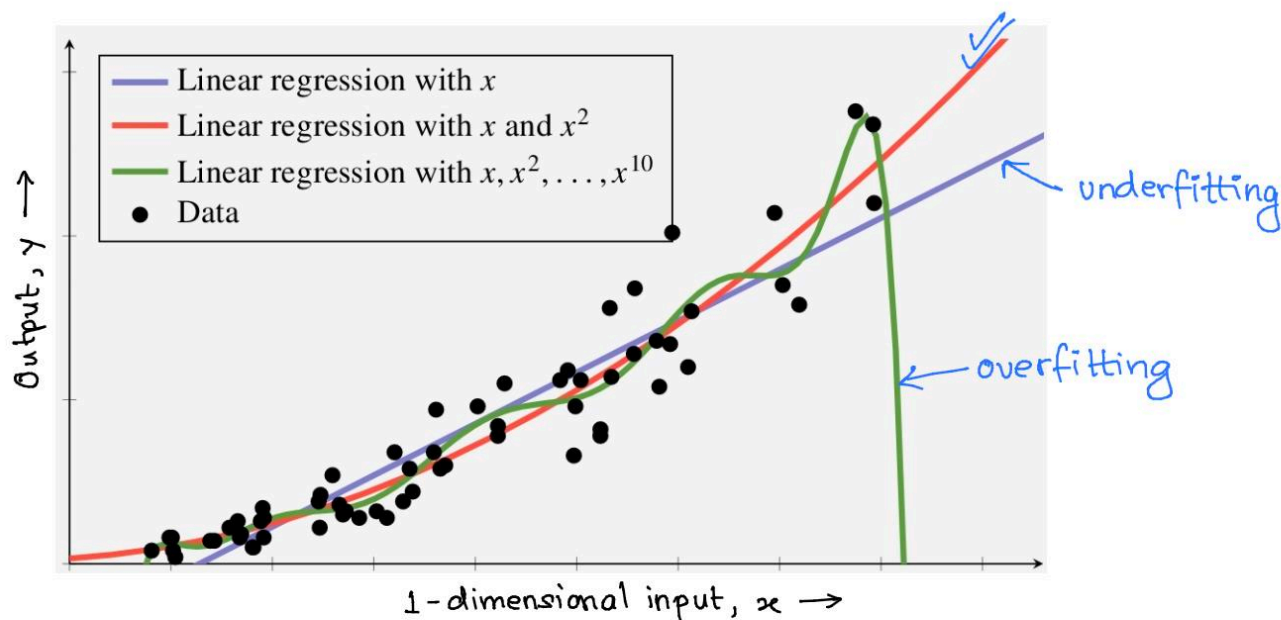
$$= \underline{x}^T \underline{\theta}$$

Still a linear model

however "lifted" the input from
one-dimension ($p=1$) to
three-dimension ($p=3$)

Polynomial Regression

- Using nonlinear transformations are quite useful in practice
 - effectively increases input dimension p
- **Downside**: Can lead to **overfitting** (the model may fit noise in the training data)



- Ways to avoid overfitting
 - Carefully select which input transformations to include
 - Use regularization
- add one inputs at a time
removing inputs that are redundant

Regularization

- **Basic idea:** Keep the parameters $\hat{\underline{\theta}}$ small unless really required!
- Meaning $\rightarrow$ if a model with small parameter values $\hat{\underline{\theta}}$ fits the data almost as well as a model with large parameter values, the model with smaller $\hat{\underline{\theta}}$ will be preferred

$$\hat{\underline{\theta}}^{(1)} = \begin{bmatrix} 0.2 \\ 1.5 \\ -0.01 \\ 0.005 \\ 0.01 \end{bmatrix}, \quad \hat{\underline{\theta}}^{(2)} = \begin{bmatrix} 2.3 \\ 10.6 \\ -1.2 \\ 0.1 \\ -1.3 \end{bmatrix}$$

both fit the data well

this set of parameters is more preferable!

- Several ways to implement the idea of "small parameter values"
 - L_0 -regularization
 - L_1 -regularization
 - L_2 -regularization (will look into this here)

L_2 Regularization

- Purpose is to prevent overfitting
- To keep $\hat{\Theta}$ small, an extra penalty term $\lambda \|\hat{\Theta}\|_2^2$ is added to the cost function
 - regularization parameter
(which is a hyper-parameter)
 - chosen by user
- Regularization parameter, $\lambda \geq 0$, controls the strength of regularization effect
 - Larger the λ value, smaller will be the values of $\hat{\Theta}$
 - $\lambda = 0$ has no effect of regularization
 - $\lambda \rightarrow \infty$ will force all parameters $\hat{\Theta}$ to 0

L_2 Regularization

- Previously studied loss function for (non-regularized) linear regression:

$$\hat{\underline{\theta}} = \underset{\underline{\theta}}{\operatorname{argmin}} \underbrace{\frac{1}{N} \|\underline{y} - \underline{X}\underline{\theta}\|_2^2}_{\text{squared loss}} \rightarrow (\underline{X}^T \underline{X}) \hat{\underline{\theta}} = \underline{X}^T \underline{y}$$

- With L_2 -regularization, add a penalty over $\underline{\theta}$ to the loss

$$\hat{\underline{\theta}} = \underset{\underline{\theta}}{\operatorname{argmin}} \left(\underbrace{\frac{1}{N} \|\underline{y} - \underline{X}\underline{\theta}\|_2^2}_{\text{tries to fit the data}} + \underbrace{\lambda \|\underline{\theta}\|_2^2}_{\text{tries to keep parameters small}} \right)$$

* Usually, the intercept parameter θ_0 is kept out of regularization

- Just like the non-regularized linear regression, the regularized problem also has a closed-form solution

$$(\underline{X}^T \underline{X} + N \lambda \underline{I}) \hat{\underline{\theta}} = \underline{X}^T \underline{y} \quad \underline{I} \leftarrow \text{identity matrix}$$

- This particular application of L_2 -regularization is called RIDGE REGRESSION